Column generation for travelling umpire problem
Advanced Computer Architecture

Yari Vanherpe

May 9, 2024



Instructor: A. Marinšek

1 Introduction

For this assignment I made a GPU-program to solve the column generation for the traveling umpire problem. For this column generation we need to find the best possible cost reduction of a path for 1 umpire.

2 Method

The cost reduction is given by: $v_u + \sum_{i \in T}(\sum_{r \in R}(a_{irs} w_{ir}))$
where T is the set of teams, R is the set of rounds, v_u and w_{ir} are shadow prices obtained from the master problem and a_{irs} is if the umpire is at team i in round r for column s.

When you calculate this every round for every (home)team, the result can be stored as a tree. My program is used to generate this tree and bound how far it goes based on the constraints for this problem. In order to accelerate the execution, I had every node calculate cost reduction and distance for all it's child nodes and than spawn new threads for each of these nodes. Once we reach a terminating node we set the return pointer that was set to the node to calculate from to this terminating node. Than the previous level checks which of the lower level nodes had the best solution and sets their return pointer to this solution.

```
void treeSearch (Node** ret) {
    index = threadIdx.x;

    if (endNode) {
        ret = this_level;
    }
    else {
        alloc (Node** next_level);
        for (team: homeTeams) {
            next_level[team] = new Node (updatedCost, updatedDistance);
        }
        treeSearch <<<1, homeTeams>>> (next_level);
        ret[index] = best (next_level);
        for (node != best (next_level) {
            delete node;
            node = nullptr;
        }
    }
}
```

This recursive calling of a GPU-function is called dynamic parallelism because you're dynamically changing how meany threads are in use.

3 Difficulties

In this section I will talk about the difficulties that occurred that eventually lead to me not finishing the project due to a lack of time. These errors can be split in 2 categories:

3.1 compiler

The first error was solved quite easily, there was an error in compilation because I used dynamic parallelism, which needs to be enabled using using the "-rdc=true" flag.

The second compiler error was occurring in the linker as a result of nvcc not correctly detecting which architecture the used GPU had. This was solved using the "-arch=sm_87" flag.

3.2 memory errors

A first major memory error that occurred was that I passed a pointer to local GPU memory instead of to global GPU memory when recursively calling my GPU function. This was quite hard to track down because this local memory was again a pointer to a globally allocated object.

Finally there were some other errors which were raised depending on what thread finished first. These errors are most probably caused by an array index out of bounds error, a null dereference or a use after free.